```
print 'Bacon strips ' . str_repeat('& bacon strips ', 3);
```

```
Output:
Bacon strips & bacon strips & bacon strips & bacon strips
```

Voilà, a foolproof method for improving any recipe. Given the simple nature (and byte-safety) of the function, it is safe to use with multi-byte encodings.

strrev()

As you might expect from its name, `strrev()` reverses the order of bytes in a string:

```
print strrev("Uno\x00 reverse!");
```

```
Output:
!esrever onU
```

Byte-safe, as we can see, but like `str_shuffle()`, there is no awareness of multi-byte encoding, and such data will be transformed to garbage.

Encoding and Escaping

Moving on from formatting and transforming, we come to the functions intended for encoding or escaping data. When string data is intended for use in another context – where it might accidentally trigger problems if it contains meta-characters for that context – it is necessary to format the string so that the results function correctly. Likewise, incoming data might contain escape sequences or encoding that prevents a program from operating correctly and would require the process to be reversed.

A character "escape" is simply a special sequence starting with a backslash \ character that maps to another character; for example, \n conventionally maps to *0x0A* (line feed), and we've seen these previously in PHP double-quoted strings. Conversely, it is necessary to sometimes indicate that meta-characters should not be interpreted as such, for example, when a PHP double-quoted string contains a double-quote character, which would otherwise terminate the string early and create a syntax error.

```
$str = "I've heard of this "programming" thing..."; // syntax error
$str = "I've heard of this \"programming\" thing..."; // correctly escaped
```